

SESIÓN

13

Aplicaciones N capas: CRUD, consultas de valor y paginación en ASP.NET Core

UNIDAD DE APRENDIZAJE 4

- CRUD de una tabla con tabla HTML, modal y AJAX (EF Core)
- Combos en cascada (ubigeo) sin recargar la página
- Consultas con valor de negocio y gráficos estadísticos
- Paginación local (Skip/Take) y con procedimientos almacenados

/ INTRODUCCIÓN

En la Sesión 12 aprendimos las piezas: **EF Core, LINQ y AJAX**. En esta sesión las juntamos en un caso completo y real de un sistema web.

- **CRUD de mantenimiento:** listar, buscar, insertar y editar registros de una tabla, con ventanas emergentes y actualización sin recargar.
- **Consultas con valor de negocio:** responder preguntas del día a día (por ejemplo, la facturación de un cliente) y mostrarlas también en **gráficos estadísticos**.
- **Paginación:** cuando hay muchos registros, mostrarlos por páginas (de 20 en 20 o 50 en 50) en lugar de todos de golpe.

Lo que cambia respecto al material clásico: el **GridView** pasa a ser una tabla HTML con EF Core, el **ModalPopup** del AJAX Toolkit pasa a ser un modal de Bootstrap con fetch, y el control **Chart** pasa a ser Chart.js. Las capas y la regla de oro siguen igual.

| / CRUD DE UNA TABLA
| CON TABLA HTML, MODAL Y AJAX

/ CRUD DE UNA TABLA

El diseño del mantenimiento (igual que en escritorio)

Un mantenimiento (CRUD) tiene siempre las mismas partes. Reproducimos en web el mismo diseño que ya hicimos en Windows Forms, aplicado a la tabla de **Proveedores**:

- **Un listado** con todos los registros (antes GridView, ahora una tabla HTML).
- **Un buscador** para filtrar por razón social.
- **Un botón "Nuevo"** que abre una ventana para registrar.
- **Un ícono de editar** en cada fila que abre la ventana con los datos cargados.

La regla de oro sigue vigente: el formulario no toca la base de datos. Llama a la capa de datos (EF Core), que hace el trabajo. Solo cambia la presentación.

/ CRUD DE UNA TABLA

Del GridView a una tabla HTML con EF Core

El material clásico usaba el control de servidor **GridView**. En ASP.NET Core listamos con una **tabla HTML** recorriendo los datos que trae EF Core:

```
<table class="table">
  <tbody>
    @foreach (var p in Model.Proveedores)
    {
      @* clase CSS condicional: rojo si esta inactivo *
      <tr class="@p.EstPrv == 0 ? "text-danger" : """>
        <td>@p.CodPrv</td>
        <td>@p.RazSocPrv</td>
        <td><button onclick="editar('@p.CodPrv')">Editar</button></td>
      </tr>
    }
  </tbody>
</table>
```

Pintar inactivos en rojo: lo que el material hacía en el evento RowDataBound ahora es una simple **clase CSS condicional** en la fila (text-danger).

/ CRUD DE UNA TABLA

Del ModalPopup del AJAX Toolkit al modal de Bootstrap

El material clásico usaba el control **ModalPopup** del AjaxControlToolkit para las ventanas de insertar y editar. Ese control ya no existe en ASP.NET Core; su reemplazo directo es el **modal de Bootstrap**, que se abre y cierra con JavaScript, sin recargar la página.

```
// se abre el modal de Bootstrap desde JavaScript, sin recargar
const modal = new bootstrap.Modal(document.getElementById("modalProveedor"));
modal.show();
```

Antes (AJAX Toolkit)	Ahora (ASP.NET Core)
ModalPopupExtender	Modal de Bootstrap (bootstrap.Modal)
PopMan01.Show()	modal.show() / modal.hide()
Grabar con postback	Grabar con fetch (sin recargar)

/ CRUD DE UNA TABLA

El buscador: filtrar con LINQ

El filtrado por razón social, que antes se hacía con **DataRowFilter**, ahora es una consulta LINQ en la capa de datos:

```
public async Task<List<Proveedor>> ListarAsync(string filtro)
{
    // si el filtro viene vacio, trae todos; si no, filtra por razon social
    return await _ctx.Proveedores
        .Where(p => filtro == "" || p.RazSocPrv.Contains(filtro))
        .OrderBy(p => p.RazSocPrv)
        .ToListAsync();
}
```

Contains se traduce a un LIKE '%filtro%' en SQL Server. EF Core arma la consulta por nosotros.

/ CRUD DE UNA TABLA

Combos en cascada (ubigeo) sin recargar

El ubigeo tiene tres combos encadenados: al elegir **Departamento** se cargan sus **Provincias**, y al elegir Provincia se cargan sus **Distritos**. El material clásico lo hacía con **AutoPostBack** (recargando). Ahora se hace con **fetch**, sin recargar:

```
// al cambiar el combo de departamento, se piden sus provincias por AJAX
cboDepartamento.addEventListener("change", async () => {
  const r = await fetch(`?handler=Provincias&dep=${cboDepartamento.value}`);
  const lista = await r.json();
  // se llena el combo de provincias con la respuesta
  llenarCombo(cboProvincia, lista);
});
```

Ventaja: la cascada ahora es instantánea y no recarga la página ni cierra el modal, a diferencia del AutoPostBack clásico.

/ CRUD DE UNA TABLA

Insertar y actualizar con EF Core

Grabar reutiliza la capa de datos, igual que en escritorio. Con EF Core, insertar y actualizar es directo:

```
// INSERTAR: se agrega la entidad y se guardan los cambios  
_ctx.Proveedores.Add(proveedor);  
await _ctx.SaveChangesAsync();
```

```
// ACTUALIZAR: se busca, se cambian sus campos y se guardan los cambios  
var p = await _ctx.Proveedores.FindAsync(codigo);  
p.RazSocPrv = proveedor.RazSocPrv;  
p.EstPrv    = proveedor.EstPrv;  
await _ctx.SaveChangesAsync(); // EF Core detecta el cambio y hace el UPDATE
```

Change Tracking: al actualizar no escribimos el UPDATE; EF Core detecta qué propiedades cambiaron y arma la sentencia por nosotros.

| / CONSULTAS DE VALOR
| Y GRÁFICOS ESTADÍSTICOS

/ CONSULTAS DE VALOR Y GRÁFICOS

Consultas con valor de negocio

- Lo más valioso de un sistema es la **información**: que responda preguntas del día a día del negocio con datos **concisos, validados y oportunos**.
- Un ejemplo clásico es la **facturación de un cliente entre dos fechas** — que ya resolvimos en la Sesión 12 con LINQ y con procedimientos almacenados.
- En web, el resultado se muestra en una tabla; y si conviene, también en un **gráfico estadístico**, porque "un gráfico vale más que mil palabras".

Nada nuevo en el acceso a datos: estas consultas usan exactamente lo de la Sesión 12 (LINQ o SP). Lo nuevo aquí es cómo **presentamos** el resultado: en tablas y en gráficos.

/ CONSULTAS DE VALOR Y GRÁFICOS

Del control Chart a Chart.js

El material clásico usaba el control **Chart** de Web Forms. Su equivalente moderno es **Chart.js**, una librería de JavaScript. Le pasamos los datos (que vienen de EF Core) y dibuja el gráfico:

```
// datos calculados en el servidor (facturacion por año) y dibujados con Chart.js
new Chart(document.getElementById("grafico"), {
  type: "bar",
  data: {
    labels: ["2023", "2024", "2025"],
    datasets: [{ label: "Total facturado (S/)", data: [10008, 47314, 75237] }]
  }
});
```

Antes (Web Forms)	Ahora (ASP.NET Core)
<asp:Chart> (control de servidor)	Chart.js (librería JavaScript)
Datos por enlace de servidor	Datos del servidor + dibujo en el cliente

| / PAGINACIÓN

/ PAGINACIÓN

¿Por qué paginar?

- Cuando una consulta devuelve muchos registros, mostrarlos **todos de golpe** satura la pantalla y hace lenta la página.
- La solución es mostrarlos por **páginas**, de 20 en 20 o de 50 en 50.

Hay dos formas de paginar, según el volumen de datos:

Paginación local (en memoria)

Se traen los registros y se muestran por páginas con **Skip/Take**. Simple, aceptable para **pocos** registros (< 1000).

Paginación con procedimiento almacenado

La base de datos devuelve **solo** la página pedida. La forma **recomendada** para muchos registros.

/ PAGINACIÓN

Paginación local con Skip / Take

Con LINQ, paginar es combinar dos operaciones: **Skip** (saltar los de las páginas anteriores) y **Take** (tomar los de esta página):

```
int tam = 20;                // tamaño de pagina
int pagina = 3;              // pagina solicitada (1, 2, 3...)

var registros = await _ctx.Proveedores
    .OrderBy(p => p.RazSocPrv)
    .Skip((pagina - 1) * tam) // salta las paginas anteriores
    .Take(tam)               // toma solo los de esta pagina
    .ToListAsync();
```

Ojo: aquí EF Core sí traduce Skip/Take a SQL (OFFSET/FETCH), así que es eficiente. Lo que NO conviene para volúmenes grandes es traer todo a memoria y paginar la lista ya cargada.

/ PAGINACIÓN

Paginación con procedimiento almacenado

Para muchos registros, lo recomendable es que la base de datos devuelva **solo la página pedida**. El SP usa **OFFSET / FETCH** y también informa el total (para saber cuántas páginas hay):

```
CREATE PROCEDURE usp_ListarFacturas_Paginacion
    @pagina INT, @tam INT
AS BEGIN
    SELECT * FROM vw_VistaFacturas
    ORDER BY Num_fac
    OFFSET (@pagina - 1) * @tam ROWS      -- salta las paginas previas
    FETCH NEXT @tam ROWS ONLY;          -- devuelve solo esta pagina
END
```

```
// se invoca desde EF Core con FromSql (entidad sin clave)
return await _ctx.FacturasPagina
    .FromSql($"EXEC usp_ListarFacturas_Paginacion {pagina}, {tam}")
    .ToListAsync();
```

La analogía de la PPT: en vez de invitar a 1500 personas a la sala y agruparlas de 50 en 50, invitas solo a las 50 que necesitas. Eso es paginar en la base de datos.

| / LABORATORIO

/ LABORATORIO

Proyecto Web_Sesion13 · sobre VentasLeon

Ampliamos el sitio de la Sesión 12 con tres ejemplos, uno por cada tema:

- **Ejemplo 1 — CRUD de Proveedores:** listado en tabla HTML, buscador, y modal de Bootstrap para insertar/editar con AJAX, incluyendo los combos de ubigeo en cascada.
- **Ejemplo 2 — Consulta de valor con gráfico:** facturación anual mostrada en tabla y en un gráfico de barras con Chart.js.
- **Ejemplo 3 — Paginación:** el listado de facturas paginado de dos formas: local (Skip/Take) y con procedimiento almacenado (OFFSET/FETCH).

Base de datos: se reutiliza VentasLeon y se agregan las tablas **Proveedor** y **Ubigeo**. El manual paso a paso acompaña a este proyecto.

/ CONCLUSIONES

- Las **capas son reutilizables**: el mismo acceso a datos (EF Core) sirve para escritorio o web; solo cambia la presentación.
- El **CRUD web moderno** usa una tabla HTML, un modal de Bootstrap y AJAX (fetch), reemplazando al GridView y al ModalPopup del AJAX Toolkit.
- Las **consultas de valor** se presentan en tablas y en gráficos (**Chart.js**), para que la información apoye la toma de decisiones.
- **Paginar** es clave con muchos registros: local con **Skip/Take** para pocos, y con **procedimiento almacenado** (OFFSET/FETCH) para volúmenes grandes.

/ BIBLIOGRAFÍA

Documentación y recursos de consulta

- **MICROSOFT — Paginación en EF Core.** Skip/Take y paginación eficiente. Recuperado de <https://learn.microsoft.com/ef/core/querying/pagination>
- **MICROSOFT — OFFSET / FETCH (Transact-SQL).** Recuperado de <https://learn.microsoft.com/sql/t-sql/queries/select-order-by-clause-transact-sql>
- **BOOTSTRAP — Modal.** Componente de ventana emergente. Recuperado de <https://getbootstrap.com/docs/5.3/components/modal/>
- **CHART.JS — Documentación.** Gráficos estadísticos en el navegador. Recuperado de <https://www.chartjs.org/docs/latest/>
- **MDN WEB DOCS — Uso de fetch.** Recuperado de https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch